# Finance Support Triage Agent

Course Name: Agentic AI

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1 | Tarang Purohit | EN22CS3011033 |
| 2 | Suhani Maheshwari | EN22CS301988 |
| 3 | Smita Kumawat | EN22CS301961 |
| 4 | Sneha Agrawal | EN22CS301963 |
| 5 | Somya Neema | EN22CS301973 |

*Group Name: Group 03D9*

*Project Number: DO-03*

*Industry Mentor Name: Mr Ashwin*

*University Mentor Name: Prof. Pradeep Baniya*

*Academic Year: 2025-26*

## 1. Problem Statement

Financial institutions and digital payment platforms receive a large number of customer support queries daily, including issues such as fraudulent transactions, duplicate charges, refund requests, and payment failures. These queries are typically handled manually by support teams, leading to delays, inconsistencies, and increased operational costs.

Manual triage requires human agents to read, understand, classify, and respond to each query, which becomes inefficient at scale. Additionally, the lack of structured processing makes it difficult to prioritize urgent issues like fraud or high-risk transactions.

Therefore, there is a need for an intelligent system that can automatically analyze user queries, extract key financial details, determine intent and urgency, and generate appropriate responses in real time.

## 2. Project Objective

The main objectives of this project are:

- To design and develop an AI-powered system for automated triage of financial support queries
- To classify user queries into categories such as fraud, refund, and payment issues
- To extract important entities like transaction amount, date, and identifiers using AI
- To assess urgency and risk level for prioritization
- To generate accurate and context-aware responses for users
- To reduce manual intervention and improve response time
- To build a scalable and modular system architecture suitable for real-world deployment

## 3. Scope of the Project

This project focuses on automating the initial handling of finance-related support queries.

- Processing user queries related to:
  - Fraud detection and alerts
  - Refund and reversal requests
  - Payment failures and transaction issues
- AI-based classification and entity extraction
- Multi-agent orchestration using CrewAI
- REST API-based backend using FastAPI
- Interactive frontend dashboard using React
- Storage and retrieval of processed data using MongoDB

## 4. Proposed Solution

The proposed solution is an **AI-driven Finance Support Triage System** that automates the processing of customer queries using a structured multi-agent pipeline.

When a user submits a query through the frontend dashboard, the request is sent to the FastAPI backend. The backend triggers a **CrewAI-based orchestration pipeline**, where multiple specialized agents work sequentially to process the query.

The system uses the **OpenRouter API** as a unified interface to access powerful language models, enabling natural language understanding and reasoning.

**Processing Flow:**

1. User Input: User submits a query via React dashboard
2. API Handling: FastAPI validates and processes the request
3. AI Orchestration: CrewAI pipeline executes multiple agents:
   - Classifier Agent: Identifies query type (fraud, refund, etc.)
   - Extractor Agent: Extracts entities like amount, date, transaction ID
   - Decision Agent: Determines risk level and required action
   - Validator Agent: Ensures structured and correct output
   - Responder Agent: Generates a user-friendly response

4. LLM Processing: Agents interact with LLMs via OpenRouter API
5. Storage: Results are stored in MongoDB
6. Response Delivery: Structured output is returned to frontend

## 5. Key Features

- Automated classification of financial queries
- Named Entity Recognition (NER) for extracting financial data
- Multi-agent architecture using CrewAI
- AI-generated contextual responses
- Asynchronous processing with status tracking
- RESTful API design for scalability
- MongoDB-based storage for audit and analytics
- Modular and extensible architecture

## 6. Overall Architecture / Workflow

The system follows a layered architecture for better separation of concerns:

**Presentation Layer**

- Built using React
- Provides UI for query input and result display

**API Layer**

- Built using FastAPI
- Handles request validation, routing, and authentication

**Orchestration Layer**

- CrewAI manages multi-agent execution
- Ensures sequential and structured processing

**AI Layer**

- Uses OpenRouter API to access language models
- Performs classification, extraction, and response generation

**Data Layer**

- MongoDB stores:
  - Queries
  - Extracted entities
  - Decisions
  - Responses

**Workflow Summary**

User → React → FastAPI → CrewAI Agents → OpenRouter (LLM) → MongoDB → Response

## 7. Tools & Technologies Used

- Frontend: React, Tailwind CSS
- **Backend:** FastAPI (Python)
- **AI Framework:** CrewAI (multi-agent orchestration)
- **LLM Access:** OpenRouter API
- **Database:** MongoDB
- **Authentication:** JWT (JSON Web Tokens)
- **Other Tools:** Uvicorn, Pydantic

## 8. Results & Output

Screenshots / Outputs

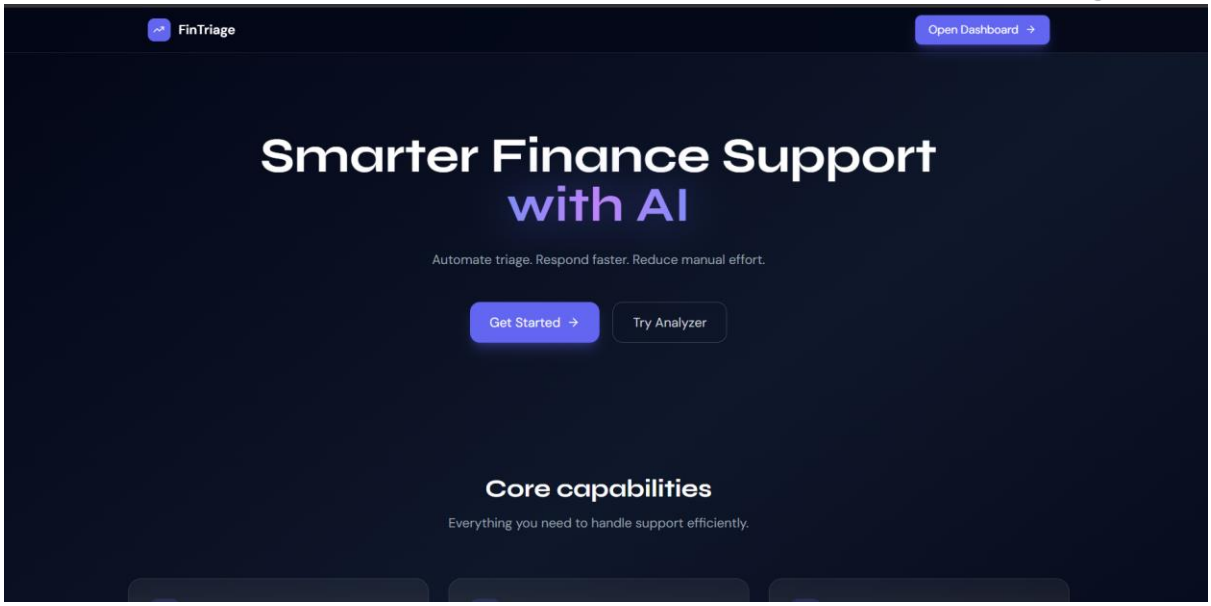


*Figure 8. 1 Landing Page – AI Finance Support Interface*

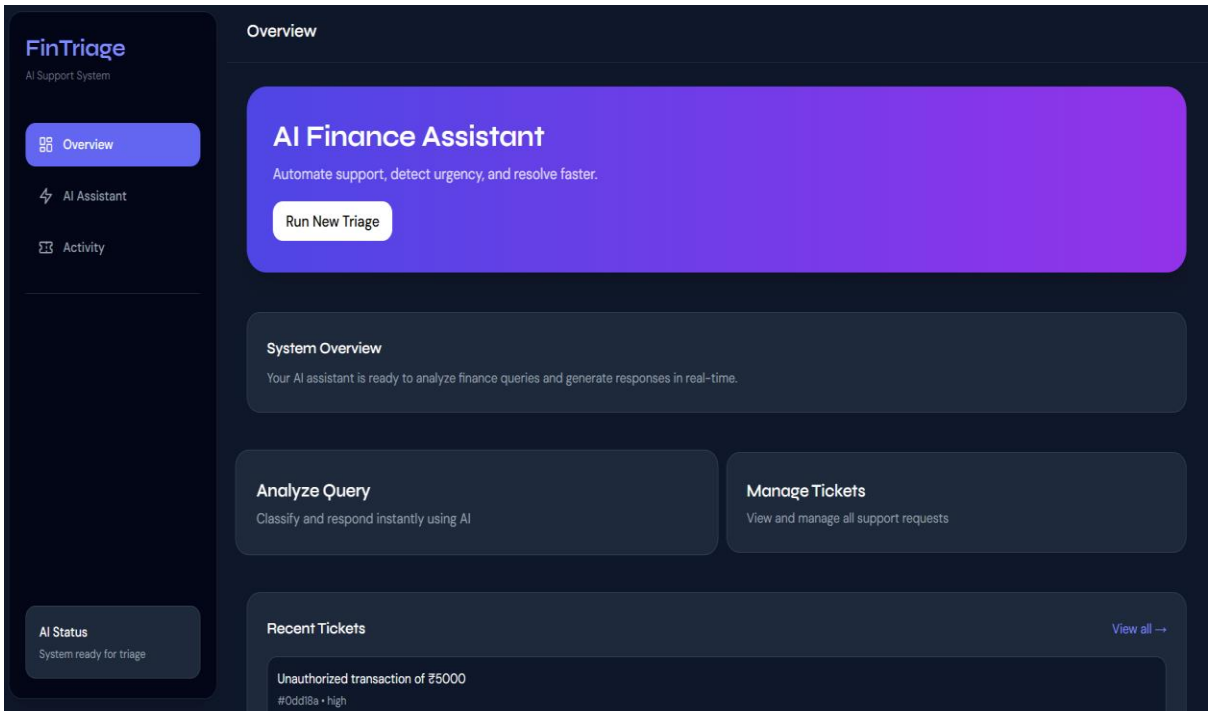


*Figure 8. 2 Dashboard Overview – AI Finance Assistant*

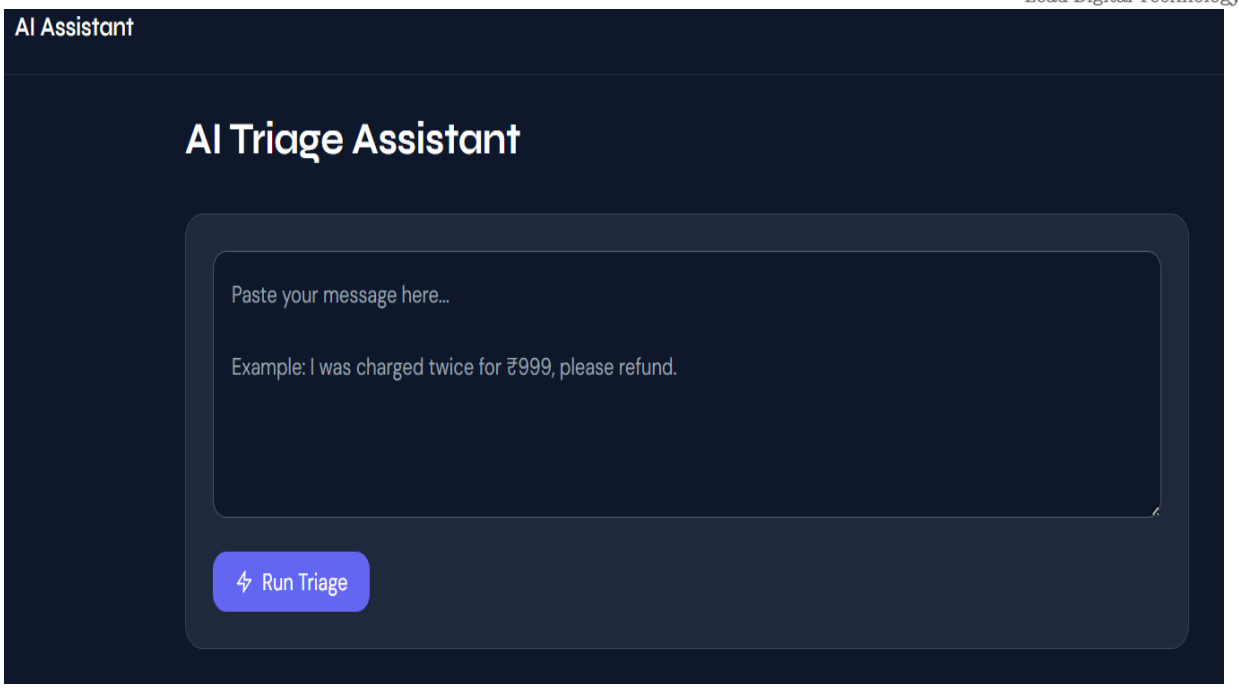


*Figure 8. 3 AI Triage Assistant – Query Input Interface*

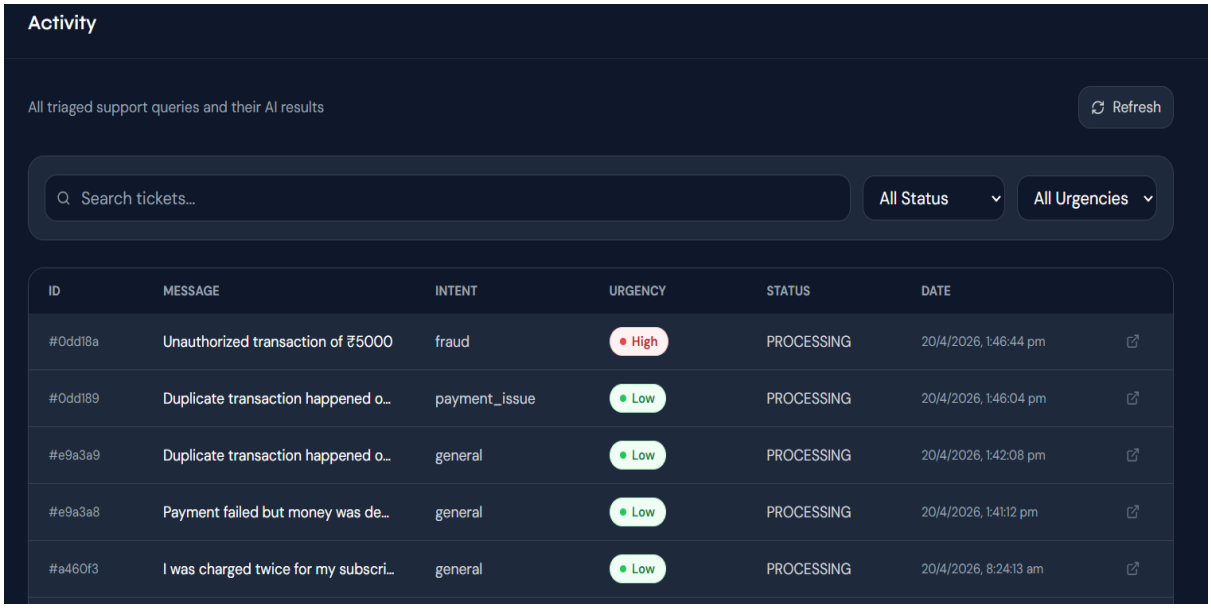


*Figure 8. 4 Activity Dashboard – Processed Tickets and Results*

**Reports / Dashboards / Models**

- Dashboard showing all processed tickets

- Structured JSON outputs from AI pipeline
- Logs for tracking system behavior

**Key Outcomes**

- Significant reduction in query processing time
- Automated classification and response generation
- Improved consistency and accuracy
- Scalable architecture suitable for real-world deployment
- Hands-on experience with AI agents and system design

## 9. Conclusion

The Finance Support Triage System successfully demonstrates the application of AI in automating customer support workflows. By leveraging a multi-agent architecture and integrating LLM capabilities through OpenRouter, the system efficiently processes financial queries and generates structured outputs.

This project highlights the potential of combining backend engineering, AI orchestration, and modern web technologies to build scalable and intelligent systems. It also provided valuable learning in system design, API development, and real-world AI integration.

## 10. Future Scope & Enhancements

- Integration with real banking and payment systems
- Advanced fraud detection using machine learning models
- Multilingual support for wider accessibility
- Voice-based query processing
- Real-time notifications (email/SMS alerts)
- Advanced analytics dashboard for insights
- Fine-tuned domain-specific AI models